I want a javascript function (in react) for an array of objects to traverse tables in the database postgres

Got it — you want a **React-side JavaScript function** that works with an **array of objects (fetched from PostgreSQL)** and helps you **traverse / analyze / display table data**.

Important clarification first:
👉 React **cannot directly access PostgreSQL**
👉 You always fetch data via an API (Node/Express, Django, etc.)
👉 Then you process it in React as an **array of objects**

---

## ✅ Example Data (from PostgreSQL via API)

```javascript
const leads = [
  { id: 1, name: "Project A", status: "Active", value: 50000, domain: "Defence" }
  { id: 2, name: "Project B", status: "Lost", value: 20000, domain: "Civil" },
  { id: 3, name: "Project C", status: "Active", value: 75000, domain: "Defence" }
];
```

JavaScript

---

## 🔥 1. Generic Traversal Function

This works like a reusable engine for any table

```javascript
export const traverseTable = (data, callback) => {
  if (!Array.isArray(data)) return [];

  return data.map((row, index) => callback(row, index));
};
```

### ✅ Usage

```
const result = traverseTable(leads, (row) => ({
  ...row,
  formattedValue: `₹${row.value.toLocaleString()}`
```

JavaScript

---

## 🔥 2. Filter + Traverse (Most Useful for Dashboards)

```
export const filterAndTraverse = (data, filters, callback) => {
  return data
    .filter((row) => {
      return Object.keys(filters).every((key) =>
        filters[key] ? row[key] === filters[key] : true
      );
    })
    .map(callback);
};
```

### ✅ Usage

```
const activeDefenceLeads = filterAndTraverse(
  leads,
  { status: "Active", domain: "Defence" },
  (row) => row
);
```

---

## 🔥 3. Aggregation Function (Analytics)

Perfect for your dashboard KPIs

```
export const aggregateTable = (data) => {
  return data.reduce(
    (acc, row) => {
      acc.totalLeads += 1;
      acc.totalValue += row.value || 0;

      if (row.status === "Active") acc.active += 1;
      if (row.status === "Lost") acc.lost += 1;
```

```
      return acc;
    },
    {
      totalLeads: 0,
      totalValue: 0,
      active: 0,
      lost: 0,
    }
  );
};
```

### ✅ Usage

```
const stats = aggregateTable(leads);

console.log(stats);
/*
{
  totalLeads: 3,
  totalValue: 145000,
  active: 2,
  lost: 1
}
*/
```

---

## 🔥 4. Group By Function (VERY IMPORTANT for Analytics UI)

```
export const groupByField = (data, field) => {
  return data.reduce((acc, row) => {
    const key = row[field];

    if (!acc[key]) acc[key] = [];
    acc[key].push(row);

    return acc;
  }, {});
};
```

### ✅ Usage

JavaScript

---

## 🔥 5. React Hook (BEST PRACTICE)

Use this to integrate with your UI

```
import { useEffect, useState } from "react";

export const useTableData = (apiUrl) => {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch(apiUrl)
      .then((res) => res.json())
      .then((result) => {
        setData(result);
        setLoading(false);
      })
      .catch(() => setLoading(false));
  }, [apiUrl]);

  return { data, loading };
};
```

---

## 🔥 6. Full Example in React Table

```
const TableComponent = ({ data }) => {
  return (
    <table>
      <thead>
        <tr>
          <th>Name</th>
          <th>Status</th>
          <th>Value</th>
        </tr>
      </thead>

      <tbody>
```

```
        {data.map((row) => (
          <tr key={row.id}>
            <td>{row.name}</td>
            <td>{row.status}</td>
            <td>₹{row.value}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
};
```

---

## 🚀 Bonus (For YOUR Project – Analytics Dashboard)

You can combine everything:

```
const processedData = filterAndTraverse(
  leads,
  { status: "Active" },
  (row) => ({
    ...row,
    valueInLakhs: row.value / 100000
  })
);

const stats = aggregateTable(processedData);
const grouped = groupByField(processedData, "domain");
```

---

## ⚡ Key Takeaway

✔ PostgreSQL → Backend API → React
✔ React handles **arrays of objects**
✔ Use:

- `map()` → traverse
- `filter()` → conditions
- `reduce()` → analytics

---

If you want next level (based on your dashboard work), I can give you:

✅ Server-side PostgreSQL query optimization
✅ Full Node.js API for this
✅ Real-time analytics (like Power BI style)
✅ Advanced KPI engine (monthly trends, FY filters)

Just tell me 👍

domestic_leads_model.js
JavaScript

DomesticLeadForm.js
JavaScript

If I want to traverse through this and use it for analytics give full clear picture and step by step instructions

Perfect — now I'll give you a **complete real-world picture using YOUR actual code** and show **how to traverse + convert it into analytics step-by-step**.

I'll connect everything:

👉 PostgreSQL (your model)
👉 Backend API
👉 React state (`orderData`)
👉 Table traversal
👉 Analytics (KPIs, charts, filters)

---

## 🔍 STEP 1: Understand YOUR Data Structure

From your model:

domestic_leads_model

Your PostgreSQL table → becomes **array of objects in React**

### Example (real structure you are getting)

```
[
  {
    tenderName: "Project A",
```

```
    tenderReferenceNo: "REF001",
    customerName: "ABC Ltd",
    civilOrDefence: "Defence",
    businessDomain: "IT",
    estimatedValueInCrWithoutGST: "10.5",
    submittedValueInCrWithoutGST: "9.8",
    wonLostParticipated: "Won",
    openClosed: "Closed",
    presentStatus: "BQ Submitted"
  }
]
```

---

## 🔍 STEP 2: Where Data Enters React

From your component:

DomesticLeadForm

```
const [orderData, setOrderData] = useState([]);

axios.get(ServerIp + API).then((response) => {
  setOrderData(response.data);
});
```

👉 This is your **main dataset**

👉 This is what you must traverse

---

## 🔥 STEP 3: Convert Raw Data → Clean Data (VERY IMPORTANT)

Your values are strings → convert to numbers

```
const cleanData = orderData.map((row) => ({
  ...row,
  estimatedValue: parseFloat(row.estimatedValueInCrWithoutGST) || 0,
  submittedValue: parseFloat(row.submittedValueInCrWithoutGST) || 0,
}));
```

---

## 🔥 STEP 4: Basic Traversal (Foundation)

```javascript
cleanData.forEach((row) => {
  console.log(row.tenderName, row.estimatedValue);
```

JavaScript

👉 This is raw traversal

👉 But analytics needs **aggregation + grouping**

---

## 🚀 STEP 5: KPI Analytics (MOST IMPORTANT)

### ✅ Total Leads

JavaScript

---

### ✅ Total Pipeline Value

```javascript
const totalPipeline = cleanData.reduce(
  (sum, row) => sum + row.estimatedValue,
  0
`
```

JavaScript

---

### ✅ Total Submitted Value

```javascript
const totalSubmitted = cleanData.reduce(
  (sum, row) => sum + row.submittedValue,
  0
`
```

JavaScript

---

### ✅ Won / Lost Count

```
const stats = cleanData.reduce(
  (acc, row) => {
    if (row.wonLostParticipated?.includes("Won")) acc.won++;
    else if (row.wonLostParticipated?.includes("Lost")) acc.lost++;
    else acc.other++;

    return acc;
  },
  { won: 0, lost: 0, other: 0 }
);
```

---

## 🚀 STEP 6: Grouping (Dashboard Charts)

### ✅ Group by Domain

```
const domainData = cleanData.reduce((acc, row) => {
  const key = row.businessDomain || "Unknown";

  if (!acc[key]) {
    acc[key] = { count: 0, value: 0 };
  }

  acc[key].count += 1;
  acc[key].value += row.estimatedValue;

  return acc;
}, {});
```

👉 Output:

```
{
  IT: { count: 5, value: 50 },
  Telecom: { count: 3, value: 20 }
`
```

---

### ✅ Group by Civil / Defence

```
const defenceSplit = cleanData.reduce((acc, row) => {
  const key = row.civilOrDefence || "Other";

  acc[key] = (acc[key] || 0) + 1;

  return acc;
}, {});
```

---

## 🚀 STEP 7: Funnel Analytics (VERY POWERFUL)

```
const funnel = cleanData.reduce(
  (acc, row) => {
    if (row.openClosed === "Open") acc.open++;
    if (row.openClosed === "Closed") acc.closed++;
    if (row.wonLostParticipated?.includes("Won")) acc.won++;

    return acc;
  },
  { open: 0, closed: 0, won: 0 }
);
```

---

## 🚀 STEP 8: Apply Filters (Your UI already has this)

You already do this:

```
const filteredData = tableData.filter((row) => {
  return row.tenderType === selectedType;
```

JavaScript

👉 Use same filtered data for analytics:

JavaScript

---

## 🚀 STEP 9: Create Reusable Analytics Engine (BEST PRACTICE)

```
export const analyzeLeads = (data) => {
  const cleanData = data.map((row) => ({
    ...row,
    estimatedValue: parseFloat(row.estimatedValueInCrWithoutGST) || 0,
  }));

  return {
    totalLeads: cleanData.length,

    totalValue: cleanData.reduce(
      (sum, row) => sum + row.estimatedValue,
      0
    ),

    won: cleanData.filter((r) =>
      r.wonLostParticipated?.includes("Won")
    ).length,

    lost: cleanData.filter((r) =>
      r.wonLostParticipated?.includes("Lost")
    ).length,

    byDomain: cleanData.reduce((acc, row) => {
      const key = row.businessDomain || "Other";
      acc[key] = (acc[key] || 0) + 1;
      return acc;
    }, {}),
  };
};
```

---

## 🚀 STEP 10: Use Inside React (IMPORTANT)

```
const analytics = analyzeLeads(orderData);
```

JavaScript

---

## 🎯 FINAL ARCHITECTURE (VERY CLEAR)

```
PostgreSQL
   ↓
Sequelize Model (your file)
   ↓
API (/getDomesticLead)
   ↓
React (orderData state)
   ↓
Traversal (map / filter / reduce)
   ↓
Analytics Engine
   ↓
Dashboard UI (cards, charts)
```

---

## 🔥 WHAT YOU ALREADY BUILT (GOOD NEWS)

From your code:

✔️ Table traversal ✅
✔️ Filtering ✅
✔️ Sorting ✅
✔️ Dynamic columns ✅

👉 You are already doing 60% of analytics

---

## 🚀 WHAT YOU SHOULD ADD NEXT

To make it **Power BI level dashboard**

I can help you add:

### 🔥 1. KPI Cards

- Total Leads
- Total Value
- Won %
- Conversion rate

### 🔥 2. Charts

- Domain distribution (Pie)
- Monthly trend (Line)
- Funnel (Bar)

### 🔥 3. Advanced Filters

- FY filter
- Owner filter
- Domain filter

---

## ⚡ SIMPLE UNDERSTANDING

👉 Traversing = looping through data
👉 Analytics = summarizing data

```
map()     → transform
filter()  → select
reduce()  → analyze (MOST IMPORTANT)
```

---

If you want next step, I'll do this for you:

✅ Convert your existing dashboard into **analytics dashboard (complete UI)**
✅ Add **charts + KPIs + filters fully integrated**
✅ Make it **production-level (like Power BI / Tableau)**

Just say:
👉 *"build full analytics dashboard from my code"* 🚀

Sources

I can help you add:

🔥 1. KPI Cards
Total Leads
Total Value
Won %
Conversion rate
🔥 2. Charts
Domain distribution (Pie)
Monthly trend (Line)
Funnel (Bar)

🔥 3. Advanced Filters
FY filter
Owner filter
Domain filter

Yes

Perfect — now we're going to **upgrade your existing system into a full analytics dashboard** 🔥
I'll give you a **complete ready-to-integrate architecture + code** based on YOUR current project.

---

## 🚀 STEP 0: What You Already Have (Important)

From your code:

- `orderData` → all leads data ✅
- Table + filters + sorting ✅
- Backend API working ✅

👉 Now we ADD:

- KPI cards
- Charts
- Advanced filters
- Analytics engine

---

## 🔥 STEP 1: Create Analytics Utility (CORE ENGINE)

👉 Create file: `analytics.js`

```
export const analyzeLeads = (data) => {
  const cleanData = data.map((row) => ({
    ...row,
    estimatedValue: parseFloat(row.estimatedValueInCrWithoutGST) || 0,
    submittedValue: parseFloat(row.submittedValueInCrWithoutGST) || 0,
    date: new Date(row.tenderDated),
  }));

  const totalLeads = cleanData.length;
```

```
const totalValue = cleanData.reduce(
  (sum, r) => sum + r.estimatedValue,
  0
);

const won = cleanData.filter((r) =>
  r.wonLostParticipated?.toLowerCase().includes("won")
).length;

const conversionRate =
  totalLeads > 0 ? ((won / totalLeads) * 100).toFixed(2) : 0;

// Domain grouping
const domainData = Object.values(
  cleanData.reduce((acc, row) => {
    const key = row.businessDomain || "Other";

    if (!acc[key]) {
      acc[key] = { name: key, value: 0 };
    }

    acc[key].value += row.estimatedValue;
    return acc;
  }, {})
);

// Monthly trend
const monthlyData = Object.values(
  cleanData.reduce((acc, row) => {
    const month = row.date.toISOString().slice(0, 7);

    if (!acc[month]) {
      acc[month] = { month, value: 0 };
    }

    acc[month].value += row.estimatedValue;
    return acc;
  }, {})
);

// Funnel
const funnel = [
  { name: "Total", value: totalLeads },
  {
    name: "Participated",
    value: cleanData.filter((r) =>
      r.wonLostParticipated?.includes("Participated")
    ).length,
  },
  { name: "Won", value: won },
];
```

```
  return {
    totalLeads,
    totalValue,
    won,
    conversionRate,
    domainData,
    monthlyData,
    funnel,
  };
};
```

---

## 🔥 STEP 2: KPI CARDS (UI)

👉 Create `KPISection.js`

```
import { Box, Paper, Typography } from "@mui/material";

const KPI = ({ title, value }) => (
  <Paper
    sx={{
      p: 3,
      borderRadius: 4,
      background: "linear-gradient(135deg,#e3f2fd,#bbdefb)",
      boxShadow: 4,
    }}
  >
    <Typography variant="subtitle2">{title}</Typography>
    <Typography variant="h5" fontWeight={700}>
      {value}
    </Typography>
  </Paper>
);

export default function KPISection({ analytics }) {
  return (
    <Box display="grid" gridTemplateColumns="repeat(4,1fr)" gap={2}>
      <KPI title="Total Leads" value={analytics.totalLeads} />
      <KPI title="Total Value (Cr)" value={analytics.totalValue.toFixed(2)} />
      <KPI title="Won Deals" value={analytics.won} />
      <KPI title="Conversion %" value={analytics.conversionRate + "%"} />
    </Box>
  );
}
```

---

# 🔥 STEP 3: CHARTS (Recharts)

👉 Install:

```bash
```

---

## 📊 1. Domain Pie Chart

```
import { PieChart, Pie, Tooltip, Cell } from "recharts";

export const DomainChart = ({ data }) => (
  <PieChart width={300} height={300}>
    <Pie data={data} dataKey="value" outerRadius={100} label />
    <Tooltip />
  </PieChart>
);
```

---

## 📈 2. Monthly Trend Line Chart

```
import { LineChart, Line, XAxis, YAxis, Tooltip } from "recharts";

export const MonthlyChart = ({ data }) => (
  <LineChart width={500} height={300} data={data}>
    <XAxis dataKey="month" />
    <YAxis />
    <Tooltip />
    <Line type="monotone" dataKey="value" />
  </LineChart>
);
```

---

## 📊 3. Funnel Chart (Bar)

```
import { BarChart, Bar, XAxis, YAxis, Tooltip } from "recharts";
```

```
export const FunnelChart = ({ data }) => (
  <BarChart width={400} height={300} data={data}>
    <XAxis dataKey="name" />
    <YAxis />
    <Tooltip />
    <Bar dataKey="value" />
  </BarChart>
);
```

---

## 🔥 STEP 4: ADVANCED FILTERS

👉 Add above your table:

```
const [fy, setFy] = useState("all");
const [owner, setOwner] = useState("all");
const [domain, setDomain] = useState("all");

const filteredData = orderData.filter((row) => {
  const year = new Date(row.tenderDated).getFullYear();

  const fyMatch = fy === "all" || year === parseInt(fy);
  const ownerMatch = owner === "all" || row.leadOwner === owner;
  const domainMatch = domain === "all" || row.businessDomain === domain;

  return fyMatch && ownerMatch && domainMatch;
});
```

---

## 🔥 STEP 5: CONNECT EVERYTHING

👉 Inside your main component:

```
import { analyzeLeads } from "./analytics";
import KPISection from "./KPISection";
import { DomainChart, MonthlyChart, FunnelChart } from "./Charts";

const analytics = analyzeLeads(filteredData);
```

---

## 🔥 STEP 6: RENDER DASHBOARD

```
<Box sx={{ mb: 4 }}>
  <KPISection analytics={analytics} />
</Box>

<Box display="flex" gap={4} flexWrap="wrap">
  <DomainChart data={analytics.domainData} />
  <MonthlyChart data={analytics.monthlyData} />
  <FunnelChart data={analytics.funnel} />
</Box>
```

---

# 🎯 FINAL RESULT (WHAT YOU GET)

## ✅ KPI Cards

- Total Leads
- Total Value
- Won Deals
- Conversion %

## ✅ Charts

- Pie → Domain distribution
- Line → Monthly trend
- Bar → Funnel

## ✅ Filters

- Financial Year
- Lead Owner
- Domain

---

# 🚀 NEXT LEVEL (I RECOMMEND)

I can upgrade this further into:

🔥 Glassmorphism UI (your theme match)
🔥 Animated KPI counters
🔥 Export analytics to Excel
🔥 Drill-down charts (click → filter table)
🔥 Real-time updates (socket.io)

---

## ⚡ FINAL UNDERSTANDING

You now have:

```
Raw Data → Traversal → Aggregation → Analytics → Visualization
```

---

If you want, next I can:

👉 Convert this into **FULL production dashboard (exact UI like your blueprint)**
👉 Integrate with your existing tabs (Overview / Pipeline / Revenue)
👉 Make it **100% ready to present/demo**

Just say:
👉 *"make it production-level dashboard"* 🚀

We are taking data from that domestic lead table in database right?

Yes — **100% correct** ✅
You are taking data from your **Domestic Leads table in PostgreSQL**.

Let me connect everything clearly using your actual setup 👇

---

## 🔗 COMPLETE DATA FLOW (YOUR PROJECT)

### 🗄️ 1. PostgreSQL Table

Defined in your model:

domestic_leads_model

👉 Table name: `DomesticLeadsModel`
👉 Columns:

- `tenderName`
- `tenderReferenceNo` (Primary Key)

- `businessDomain`
- `estimatedValueInCrWithoutGST`
- `wonLostParticipated`
- `openClosed`
- etc.

---

## ⚙️ 2. Backend (Node + Sequelize)

Your API:

JavaScript

👉 This fetches data from PostgreSQL:

JavaScript

👉 Returns JSON like:

```
[
  { tenderName: "A", businessDomain: "IT", estimatedValueInCrWithoutGST: "10" }
```

JavaScript

---

## 🌐 3. React Frontend (YOUR CODE)

From your component:

DomesticLeadForm

```
axios.get(ServerIp + API).then((response) => {
  setOrderData(response.data);
```

JavaScript

👉 This means:

```
PostgreSQL → API → React state (orderData)
```

---

## 🔥 KEY POINT (VERY IMPORTANT)

👉 `orderData` = **your entire database table (in array form)**

JavaScript

Each item:

JavaScript

---

## 🔍 WHAT YOU ARE DOING NOW

In your table:

JavaScript

👉 You are already **traversing database data**

---

## 🚀 WHAT WE ADDED (ANALYTICS)

Now instead of just displaying:

JavaScript

We do:

```
orderData → filter → traverse → analyze → charts + KPIs
```

---

## 🔥 SIMPLE FLOW (REMEMBER THIS)

```
Database (Postgres)
   ↓
Sequelize Model
   ↓
API (/getDomesticLead)
   ↓
React (orderData)
   ↓
map / filter / reduce
   ↓
Analytics Dashboard
```

---

## 🧠 VERY CLEAR UNDERSTANDING

### ❓ Are we directly querying database in React?

❌ NO

### ❓ Are we using database data?

✅ YES

👉 Through API

---

## 🔥 REAL EXAMPLE (FROM YOUR PROJECT)

JavaScript

👉 This means:

✔ You are analyzing **real database data**
✔ Not dummy data
✔ Not static data

---

## ⚠️ IMPORTANT IMPROVEMENT (RECOMMENDED)

Currently:

JavaScript

👉 Convert in backend or frontend:

JavaScript

---

## 🚀 FINAL ANSWER

✅ YES — You are taking data from **Domestic Leads PostgreSQL table**
✅ React gets it via API
✅ Then you traverse it using `map`, `filter`, `reduce`
✅ Then convert it into analytics

---

## 💡 NEXT STEP (POWER MOVE)

If you want even better:

👉 Move some analytics to backend:

SQL

```
SELECT sum(value) FROM leads;
```

👉 Faster + scalable

---

If you want, next I can:

✅ Build backend analytics APIs (Postgres optimized)
✅ Add caching (fast dashboard)
✅ Add real-time updates

Just say 👍

Sources